

AI Traffic Brain

Intelligent Urban Traffic Management System

TECHNICAL REPORT — VERSION 1

Complete Development History & Technical Documentation

Mohammed Amine ACHAAR
Mohamed ZAKANE

Supervisor: Dr. EN-NOUAARY ABDESLAM

Institut National des Postes et Télécommunications
Rabat, Morocco
Academic Year 2025–2026

Contents

1	Introduction	4
1.1	Project Context	4
1.2	Personal Motivations	4
1.3	Initial Objectives	4
2	System Architecture	6
2.1	Technology Stack	6
2.2	Project Structure	6
2.3	Why These Technology Choices	7
2.3.1	Python 3.13	7
2.3.2	SUMO vs Other Simulators	7
2.3.3	Streamlit vs Flask/Django	7
2.3.4	Why NOT Pandas for CSV	8
3	The Four Controllers — Detailed Implementation	9
3.1	Controller 1: Fixed	9
3.1.1	Implementation	9
3.1.2	Why This is Inefficient	10
3.2	Controller 2: Smart	10
3.2.1	Adaptive FSM with Pressure Logic	10
3.2.2	Pressure Formula	10
3.2.3	Switching Logic	10
3.2.4	Green Wave Coordination	11
3.2.5	Result	11
3.3	Controller 3: Vision	11
3.3.1	Architecture	11
3.3.2	3-Filter Safety System	12
3.3.3	Emergency Override Logic	12
3.3.4	Result	13
3.4	Controller 4: Reinforcement Learning	13
3.4.1	State Encoding Evolution	13
3.4.2	Reward Function Evolution	14
3.4.3	Action Space	15
3.4.4	Training Configuration	16
3.4.5	Pretraining Strategy	16
3.4.6	Anti-Oscillation Guard	16
3.4.7	Final Result	17
4	Major Problems Encountered & Solutions	18
4.1	Problem 1: State Aliasing Bug	18
4.1.1	Symptoms	18
4.1.2	Debugging Process	18
4.1.3	Final Solution	18
4.2	Problem 2: Reward Drift	18
4.2.1	Symptoms	18
4.2.2	Attempts (5+ Iterations)	19

4.2.3	Final Solution	19
4.3	Problem 3: Q-Table File Corruption	19
4.3.1	Symptoms	19
4.3.2	Attempts	19
4.3.3	Final Solution	20
4.4	Problem 4: TraCI Version Incompatibilities	20
4.4.1	Symptoms	20
4.4.2	Solution	20
4.5	Problem 5: Dashboard Bugs	21
4.5.1	Pandas DLL Security Block	21
4.5.2	RL Results Not Displayed	21
4.5.3	Light Theme Applied by Mistake	21
4.6	Problem 6: Live Map Redesign Failures	21
4.6.1	Context	21
4.6.2	Attempts	21
4.6.3	Solution	21
5	Final Results	22
5.1	Performance Analysis	22
5.1.1	Why RL Beat All Others	22
5.1.2	Why Smart Failed	22
5.1.3	Why Vision Failed	23
5.2	Statistical Significance	23
6	Lessons Learned	24
6.1	Technical Skills Acquired	24
6.2	Workflow Insights	25
6.2.1	Split AI Workflow	25
6.3	What We'd Do Differently	25
7	Conclusion	27
7.1	Key Achievements	27
7.2	Development Statistics	28
7.3	Future Work	28
7.4	Final Thoughts	29

1 Introduction

This document provides a **complete technical history** of the AI Traffic Brain project, including every decision, problem, iteration, and solution encountered during development. This version is intended as a personal archive and technical reference for future work.

1.1 Project Context

The rapid urban growth in Moroccan cities, particularly **Casablanca** and **Rabat**, has led to severe traffic congestion. Current traffic management relies primarily on:

- **Fixed-time traffic lights:** 30-second cycles regardless of actual traffic
- **Manual intervention:** Police officers directing traffic during peak hours
- **No emergency vehicle priority:** Ambulances stuck in congestion

These systems cannot adapt to real-time conditions, resulting in:

- Average waiting times exceeding 3-5 minutes at major intersections
- Emergency response delays of 10-15 minutes in dense areas
- Fuel waste and increased pollution from idling vehicles
- Economic losses from reduced mobility

1.2 Personal Motivations

We chose this project to achieve the following learning objectives:

1. **Master reinforcement learning:** Implement Q-learning from scratch, not just use libraries
2. **Work with realistic simulation:** Use SUMO (industry-standard traffic simulator)
3. **Build complete system:** From simulation to AI to dashboard
4. **Address real problem:** Urban traffic is a critical issue in Morocco
5. **Create deployable solution:** Version 2 will use real Casablanca locations

1.3 Initial Objectives

At project start (September 2025), we defined these goals:

Success Criteria
1. Create realistic urban traffic simulation with 3 intersections
2. Implement at least 3 different control strategies
3. Train RL agent that outperforms fixed-time baseline by $\geq 10\%$

4. Build interactive dashboard for visualization
5. Demonstrate emergency vehicle priority handling
6. Complete by end of semester (January 2026)

Spoiler: All objectives achieved. RL beat baseline by 12.7%.

2 System Architecture

2.1 Technology Stack

Table 1 shows the complete technology stack with exact versions.

Table 1: Technology Stack — All Components

Component	Technology / Version
Language	Python 3.13
Simulation Engine	SUMO 1.26 (Simulation of Urban Mobility)
SUMO-Python Bridge	TraCI module (built-in)
Dashboard Framework	Streamlit 1.40
Visualization	Plotly 5.24
RL Algorithm	Tabular Q-learning (custom implementation)
Data Storage	CSV (native <code>csv</code> module), Pickle (Q-tables)
Vision Component	Simulated YOLO detection pipeline
Development Machine	Windows 11, Intel i7-12700 (8 cores), 16GB RAM
AI Workflow	Claude Code (implementation) + ChatGPT (planning)
Version Control	Git 2.47, GitHub (public repo)

2.2 Project Structure

Final directory structure after all iterations:

```

1 AI-Traffic-Brain/
2     main.py                # CLI entry      7 modes
3     core/
4         traffic_env.py      # SUMO/TraCI wrapper
5         fixed_controller.py  # Fixed 30s/5s cycle
6         smart_controller.py  # Demand-adaptive FSM
7         rl_controller.py     # Q-learning (v6 state)
8         vision_bridge.py     # Smart + camera override
9         camera_detector.py   # Simulated YOLO
10    simulation/
11        simulation.sumocfg    # 3600s, 1s steps
12        intersection.net.xml  # Compiled network
13        intersection.nod.xml  # 9 nodes (3 TL + 6 stubs)
14        intersection.edg.xml  # 16 edges
15        routes.rou.xml       # Vehicles, flows, accidents
16    dashboard/
17        app.py                # Main Streamlit dashboard
18        map_view.py           # Folium city map
19        pages/live_map.py     # Animated HTML/JS schematic
20    data/
21        results_fixed.csv
22        results_smart.csv
23        results_vision.csv
24        results_rl.csv
25        qtable.pkl            # Active Q-table (10,832 entries)
26        qtable.bak            # Atomic backup

```

Listing 1: Project Directory Tree

2.3 Why These Technology Choices

2.3.1 Python 3.13

- Native SUMO/TraCI support (no bindings needed)
- Rich ecosystem: Streamlit, Plotly, NumPy all mature
- Fast prototyping for RL experiments (no compilation)
- Both team members proficient in Python
- **Alternative considered:** C++ (rejected — too slow for prototyping)

2.3.2 SUMO vs Other Simulators

Table 2: Traffic Simulator Comparison

Simulator	Cost	RL Support	Decision
SUMO	Free	TraCI API	Selected
VISSIM	\$\$\$	Limited	Too expensive
AIMSun	\$\$	Good	License issues
Custom (from scratch)	Free	Full control	Too time-consuming

SUMO advantages:

- Open-source, active community (2000+ citations/year)
- TraCI API enables step-by-step control
- Can import OpenStreetMap networks (future: real Casablanca)
- Realistic car-following models (Krauss model)

2.3.3 Streamlit vs Flask/Django

Decision: Streamlit for rapid dashboard development.

Reasoning:

- No HTML/CSS needed — pure Python
- Built-in state management (`st.session_state`)
- Native Plotly integration
- Sufficient for academic demo
- **Trade-off:** Less control than Flask, but 10× faster development

2.3.4 Why NOT Pandas for CSV

Problem encountered: Windows DLL security blocked Pandas import in Streamlit.

Solution: Rewrote all CSV handling using Python's built-in `csv` module.

Code example:

```
1 import csv
2
3 def load_results(filepath):
4     data = []
5     with open(filepath, 'r', encoding='utf-8') as f:
6         reader = csv.DictReader(f)
7         for row in reader:
8             data.append({
9                 'time': int(row['time']),
10                'avg_wait': float(row['avg_wait']),
11                'queue': int(row['queue'])
12            })
13     return data
```

Listing 2: CSV Loading Without Pandas

This worked perfectly — no dependencies, no DLL issues.

3 The Four Controllers — Detailed Implementation

This section documents the complete implementation of all four traffic controllers, including **exact parameter values**, code snippets, and design rationale.

3.1 Controller 1: Fixed

3.1.1 Implementation

The simplest baseline: all 3 intersections synchronized to the same 70-second cycle.

Phase durations:

- NS_GREEN = 30s
- NS_YELLOW = 5s
- EW_GREEN = 30s
- EW_YELLOW = 5s
- **Total cycle:** 70 seconds

Synchronization: All intersections advance phases together (no offset).

```

1 class FixedController:
2     PHASE_DURATION = {
3         'NS_GREEN': 30,
4         'NS_YELLOW': 5,
5         'EW_GREEN': 30,
6         'EW_YELLOW': 5
7     }
8
9     def __init__(self, tl_ids):
10         self.tl_ids = tl_ids # ['int_A', 'int_B', 'int_C']
11         self.phase = 'NS_GREEN'
12         self.timer = 0
13
14     def step(self, env):
15         self.timer += 1
16
17         if self.timer >= self.PHASE_DURATION[self.phase]:
18             self.timer = 0
19             self.phase = self._next_phase(self.phase)
20
21             # Apply to all intersections
22             for tl_id in self.tl_ids:
23                 env.set_phase(tl_id, self.phase)
24
25     def _next_phase(self, current):
26         cycle = ['NS_GREEN', 'NS_YELLOW', 'EW_GREEN', 'EW_YELLOW']
27         idx = cycle.index(current)
28         return cycle[(idx + 1) % 4]

```

Listing 3: Fixed Controller Core Logic

3.1.2 Why This is Inefficient

Fixed Controller Inefficiency

Problem 1: During low-traffic periods (2 AM), vehicles still wait 30s even if cross-street is empty.

Problem 2: During high-traffic (rush hour), 30s is insufficient to clear queues $\Rightarrow$ perpetual red.

Problem 3: No emergency vehicle priority — ambulances wait full cycle.

Result: 210.4s average waiting time (baseline).

3.2 Controller 2: Smart

3.2.1 Adaptive FSM with Pressure Logic

The Smart controller measures traffic **pressure** on each axis and switches when competing axis has significantly higher demand.

Key parameters (all empirically tuned):

Table 3: Smart Controller Parameters

Parameter	Value
MIN_GREEN	8 seconds (prevents oscillation)
YELLOW_DURATION	3 seconds
MAX_GREEN_NORMAL	60 seconds
MAX_GREEN_CONGESTION	90 seconds (if wait > 500s)
SWITCH_RATIO	1.5 (competing / current pressure)
EMERGENCY_HOLD	60 seconds
WAVE_WINDOW	± 6 seconds around ETA

3.2.2 Pressure Formula

$$\text{pressure} = \frac{\text{vehicle_count}}{\text{max_seen_count}} + \frac{\text{waiting_time}}{\text{max_seen_wait}} \quad (1)$$

Both components normalized to $[0, 1]$.

3.2.3 Switching Logic

```

1 def should_switch(self, current_axis, competing_axis):
2     if self.green_timer < self.MIN_GREEN:
3         return False # Anti-oscillation
4
5     if self.green_timer >= self.MAX_GREEN_NORMAL:
6         return True  # Fairness cap
7
8     current_pressure = self.compute_pressure(current_axis)
9     competing_pressure = self.compute_pressure(competing_axis)
10
11     ratio = competing_pressure / (current_pressure + 1e-6)

```

```

12
13     return ratio >= self.SWITCH_RATIO # 1.5x threshold

```

Listing 4: Smart Controller Switch Decision

3.2.4 Green Wave Coordination

Concept: When int_A switches to EW green, schedule int_B to switch to EW green after estimated travel time.

Implementation:

```

1 # When int_A switches to EW_GREEN
2 if intersection == 'int_A' and new_phase == 'EW_GREEN':
3     travel_time = DISTANCE_A_TO_B / AVERAGE_SPEED # 400m / 40 km/h = 36
4     s
5     # Schedule int_B to switch at t + 36s ( 6s window)
6     self.schedule_wave('int_B', 'EW_GREEN', time + travel_time, window
    =6)

```

Listing 5: Green Wave Logic

Why this helps: Vehicles traveling from A to B hit green lights instead of stopping.

3.2.5 Result

Average waiting time: 213.5 seconds (+1.5% worse than Fixed).

Why worse?

- Switching overhead: Yellow phases add delay
- Green wave coordination sometimes extends greens when not optimal
- Hardcoded SWITCH_RATIO=1.5 is not always the right threshold
- Smart “thinks” it’s helping but lacks global optimization

3.3 Controller 3: Vision

3.3.1 Architecture

Vision = Smart controller + simulated camera detection.

Camera placement: One virtual camera per intersection (3 total).

Detection pipeline:

1. SUMO provides vehicle list with types
2. Simulated YOLO assigns confidence scores:
 - Emergency vehicles: 97–99% confidence
 - Normal vehicles: 85–92% confidence
 - False positives: 5% background noise
3. 3-filter safety system (see below)
4. Override Smart controller if emergency detected

3.3.2 3-Filter Safety System

Vision Safety Filters

Filter 1: Cooldown

- No emergency override within 300s of last override at same intersection
- Prevents oscillation from false positives

Filter 2: Confirmation

- Require 3 consecutive frames with confidence $\geq 97\%$
- 1-second steps $\Rightarrow$ 3 consecutive = 3 seconds of detection
- Eliminates transient false positives

Filter 3: Night Mode

- During low traffic (22:00–06:00), extend green by 15s instead of full override
- Reduces disruption when roads are mostly empty

3.3.3 Emergency Override Logic

```

1 def check_emergency_override(self, intersection, detections):
2     # Filter 1: Cooldown
3     if time.time() - self.last_override[intersection] < 300:
4         return False
5
6     # Filter 2: Confirmation (3 consecutive frames)
7     high_conf = [d for d in detections if d['confidence'] >= 0.97
8                  and d['class'] == 'emergency']
9
10    if len(high_conf) > 0:
11        self.confirm_buffer[intersection].append(True)
12    else:
13        self.confirm_buffer[intersection].clear()
14
15    if len(self.confirm_buffer[intersection]) < 3:
16        return False
17
18    # Filter 3: Night mode (22:00 - 06:00)
19    if self.is_night_mode():
20        self.extend_green(intersection, 15) # Soft override
21        return False
22
23    # Full override: immediate green, hold 60s
24    self.force_green(intersection, axis='emergency_direction', hold=60)
25    self.last_override[intersection] = time.time()
26    return True

```

Listing 6: Vision Emergency Override

3.3.4 Result

Average waiting time: 325.3 seconds (+54.6% worse than Fixed).

Why much worse?

1. Emergency overrides disrupt normal traffic flow significantly
2. Even with 300s cooldown, 15 emergencies across 3600s $\Rightarrow$ frequent disruptions
3. 60-second hold creates cascading delays on cross-streets
4. False positives (even at 5%) add unnecessary switches
5. Vision overhead (camera checks every step) slows decision-making

Lesson: Emergency priority is *critical for safety* but *terrible for average metrics*. This is expected and acceptable.

3.4 Controller 4: Reinforcement Learning

This section documents the **complete RL development journey**, including all failed attempts.

3.4.1 State Encoding Evolution

We went through **3 major state versions** before finding a working design.

Version 4 (Initial Attempt) State features: 13 per-lane vehicle counts

- int_A: NS_left, NS_straight, NS_right, EW_left, EW_straight, EW_right
- int_B: (same 6 lanes)
- int_C: NS_straight only (T-junction, 1 lane)
- Total: 13 features

State space: Assuming max 20 vehicles per lane:

$$|\mathcal{S}| = 20^{13} \approx 7 \times 10^{16} \text{ states} \quad (2)$$

Problem: Too sparse. After 50 training episodes (180,000 steps), only 2,341 states visited $\Rightarrow$ 99.9999% of Q-table empty.

Result: Random performance, no learning visible.

Version 5 (Axis Aggregation) State features: 8 aggregate NS/EW queues

- int_A: A_NS (sum of NS lanes), A_EW (sum of EW lanes)
- int_B: B_NS, B_EW
- int_C: C_NS, C_EW
- Emergency: 0 or 1 (binary flag)

- Roundabout occupancy: B_inside (vehicles inside int_B)
- Total: 8 features

State space: Max 50 vehicles per axis:

$$|\mathcal{S}| = 50^6 \times 2 \times 20 \approx 2.3 \times 10^{10} \text{ states} \quad (3)$$

Much better! But...

Problem: State aliasing. Same traffic, different phases encoded identically.

Example bug:

- State: (A_NS=5, A_EW=3, ..., emergency=0) with int_A phase = NS_GREEN
- State: (A_NS=5, A_EW=3, ..., emergency=0) with int_A phase = EW_GREEN
- $\Rightarrow$ **Same state encoding!**

Result: Contradictory Q-value updates. Training unstable.

Version 6 (Final — With Phase Bits) State features: 11 total

- Traffic: 8 features (same as v5)
- Phase bits: 3 features (A_phase, B_phase, C_phase)
 - 0 = NS green, 1 = EW green

State space:

$$|\mathcal{S}| = 50^6 \times 2 \times 20 \times 2^3 = 23,328 \text{ states} \times 8 \text{ actions} = 186,624 \text{ max Q-entries} \quad (4)$$

Solution: Different phases = different states $\Rightarrow$ no more aliasing.

Result: Stable learning. After 100 episodes, Q-table has 10,832 entries (5.8% of state-action space), sufficient for convergence.

3.4.2 Reward Function Evolution

This was the **hardest part** — we iterated **5+ times** on the reward function.

Version 1 (Naive)

$$r = - \sum_{\text{all vehicles}} \text{waiting_time}_i \quad (5)$$

Problem: Reward magnitude explodes with traffic volume. High-traffic states get huge negative rewards not because controller is bad, but because there are more vehicles.

Example:

- Low traffic (10 vehicles, avg 20s wait): $r = -200$
- High traffic (100 vehicles, avg 20s wait): $r = -2000$
- $\Rightarrow$ RL thinks high traffic = bad action, avoids those states

Version 2 (Added Throughput)

$$r = 10 \times \text{cars_passed} - 0.05 \times \sum \text{waiting_time} \quad (6)$$

Problem: Still penalizes traffic volume. The 0.05 coefficient didn't fix the drift — just scaled it.

Version 3 (Removed Sum Penalty)

$$r = 10 \times \text{cars_passed} - 1.0 \times \text{total_halting} \quad (7)$$

Problem: Penalizes halting vehicles even when they're stopped at red lights (unavoidable). RL tried to minimize halting by keeping all lights green simultaneously, which is impossible.

Version 4 (Final — Advantage Normalization)

$$\begin{aligned} r_{\text{raw}} = & 10 \times \text{throughput} + 15 \times \text{queue_improvement} - 5 \times \text{overflow} \\ & - 10 \times \text{starvation} + 100 \times \text{emergency_cleared} \\ & - 0.1 \times \text{queue_penalty} - 2.0 \times \text{switch_penalty} \end{aligned} \quad (8)$$

Then: Normalize by **advantage**:

$$r_{\text{delta}} = \frac{r_{\text{raw}} - \text{moving_avg}(r_{\text{last } 100 \text{ steps}})}{100} \quad (9)$$

This removes absolute reward drift and focuses RL on relative improvement.

Result: Stable convergence. Reward variance drops by 10× after normalization.

3.4.3 Action Space

Joint actions: All 3 intersections controlled simultaneously.

Table 4: RL Action Space (8 Actions)

Action ID	int_A	int_B	int_C
0	NS	NS	NS
1	NS	NS	EW
2	NS	EW	NS
3	NS	EW	EW
4	EW	NS	NS
5	EW	NS	EW
6	EW	EW	NS
7	EW	EW	EW

Why joint? Allows RL to learn green wave coordination automatically (e.g., action 6 = all EW green).

3.4.4 Training Configuration

Table 5: RL Hyperparameters (Final)

Parameter	Value
Learning rate (α)	0.1
Discount factor (γ)	0.95
Exploration (initial ϵ)	100% (episode 1)
Exploration decay	0.98 per episode
Exploration (final ϵ)	1% (episode 100)
Pretraining	Imitation from Smart controller
Pretraining Q-value	+5.0 for Smart actions
Episodes	100 (converged at episode 87)
Steps per episode	3600 (1 hour simulation)
Total training steps	360,000
Q-table size	10,832 entries (5.8% of max)

3.4.5 Pretraining Strategy

Problem: Pure exploration too slow (random actions terrible at traffic control).

Solution: **Imitation pretraining.**

1. Run Smart controller for 10 episodes
2. Record (state, action) pairs
3. Initialize Q-table: $Q(s, a_{\text{smart}}) = +5.0$ for actions Smart would take
4. Start RL training from this warm-started Q-table

Result: Convergence accelerated from 150+ episodes to 87 episodes.

3.4.6 Anti-Oscillation Guard

Problem: RL sometimes switches phases every step (oscillation).

Solution: Enforce `MIN_GREEN = 8 seconds`.

```

1 def apply_action(self, action, current_phases):
2     new_phases = self.decode_action(action) # [NS, EW, NS]
3
4     for i, (tl_id, new_phase, current_phase) in enumerate(
5         zip(self.tl_ids, new_phases, current_phases)
6     ):
7         if new_phase != current_phase:
8             # Check if green timer >= 8s
9             if self.green_timers[i] < 8:
10                 new_phases[i] = current_phase # Block switch
11
12     return new_phases

```

Listing 7: Anti-Oscillation Logic

3.4.7 Final Result

Average waiting time: 183.8 seconds (-12.7% vs Fixed baseline).

Why RL won:

1. Learned optimal green duration for each traffic state
2. No hardcoded thresholds (unlike Smart's $1.5\times$ ratio)
3. Balanced throughput + fairness automatically via reward
4. Emergency priority without excessive disruption
5. Pretraining bootstrapped learning from Smart's knowledge

4 Major Problems Encountered & Solutions

This section documents every significant problem, how many times we tried to fix it, and the final solution.

4.1 Problem 1: State Aliasing Bug

4.1.1 Symptoms

Q-values for the same state would update in contradictory directions:

- Episode 10: $Q(s, a = 0) = +15.3$
- Episode 20: $Q(s, a = 0) = -8.7$
- Episode 30: $Q(s, a = 0) = +22.1$

Training was unstable — no convergence visible.

4.1.2 Debugging Process

1. **Attempt 1:** Reduced state space ($v4 \rightarrow v5$) — didn't help
2. **Attempt 2:** Tried different learning rates ($\alpha = 0.01, 0.05, 0.2$) — no improvement
3. **Attempt 3:** Added logging to track state collisions
 - Discovered: Same state tuple appearing with different optimal actions
 - Example: $(5, 3, 2, 4, 8, 6, 0, 15) \rightarrow$ both action 0 (NS) and action 4 (EW) optimal
 - **Hypothesis:** Missing information in state
4. **Attempt 4:** Added current phase to state ($v6$) — **SOLVED**

4.1.3 Final Solution

Added 3 phase bits to state: (`traffic_features`, `A_phase`, `B_phase`, `C_phase`).

Now $(traffic = X, phase = NS)$ and $(traffic = X, phase = EW)$ are different states.

Result: State aliasing completely eliminated. Training stabilized.

4.2 Problem 2: Reward Drift

4.2.1 Symptoms

Rewards would explode to -5000 or $+3000$ depending on traffic volume:

Episode 10: Avg reward = -1245.3 (low traffic)
Episode 20: Avg reward = -4873.2 (rush hour)
Episode 30: Avg reward = -987.6 (low traffic)

Figure 1: Reward drift example (before normalization)

Q-learning couldn't handle this scale variation.

4.2.2 Attempts (5+ Iterations)

1. **Attempt 1:** Normalize by /1000 — helped a bit, but not enough
2. **Attempt 2:** Normalize by /100 — similar result
3. **Attempt 3:** Remove normalization entirely — much worse
4. **Attempt 4:** Subtract baseline (mean reward over all episodes) — unstable (baseline changes)
5. **Attempt 5:** Advantage normalization (moving avg of last 100 steps) — **SOLVED**

4.2.3 Final Solution

```
1 class RewardNormalizer:
2     def __init__(self, window=100):
3         self.history = deque(maxlen=window)
4         self.mean = 0.0
5
6     def normalize(self, reward):
7         self.history.append(reward)
8         self.mean = sum(self.history) / len(self.history)
9
10        # Advantage: how much better than recent average
11        delta = reward - self.mean
12
13        # Scale to reasonable range
14        return delta / 100.0
```

Listing 8: Advantage Normalization

Result: Reward variance dropped by 10×. Convergence achieved.

4.3 Problem 3: Q-Table File Corruption

4.3.1 Symptoms

After a crash or Ctrl+C during training, Q-table file unloadable:

```
Traceback (most recent call last):
  File "main.py", line 142, in <module>
    q_table = pickle.load(f)
_pickle.UnpicklingError: invalid load key, '\x00'
```

Lost hours of training multiple times.

4.3.2 Attempts

1. **Attempt 1:** Save more frequently (every 10 steps) — didn't prevent corruption
2. **Attempt 2:** Try-except around `pickle.load` — couldn't recover corrupted file
3. **Attempt 3:** Atomic write with temp file + `os.replace()` — **SOLVED**

4.3.3 Final Solution

```

1 import pickle
2 import os
3 import shutil
4
5 def save_q_table(q_table, filepath):
6     # Step 1: Write to temporary file
7     tmp_path = filepath + '.tmp'
8     with open(tmp_path, 'wb') as f:
9         pickle.dump(q_table, f)
10
11     # Step 2: Verify it's complete (optional but recommended)
12     with open(tmp_path, 'rb') as f:
13         test_load = pickle.load(f) # Will raise if corrupted
14
15     # Step 3: Backup old file
16     if os.path.exists(filepath):
17         shutil.copy(filepath, filepath + '.bak')
18
19     # Step 4: Atomic replace
20     os.replace(tmp_path, filepath) # Atomic on POSIX and Windows

```

Listing 9: Atomic Q-Table Write

Guarantees:

- File is never in half-written state
- If crash during write, old file still valid
- .bak file as secondary backup

4.4 Problem 4: TraCI Version Incompatibilities

4.4.1 Symptoms

After upgrading SUMO 1.20 → 1.26, code broke:

`AttributeError: module 'traci.constants' has no attribute 'VAR_LANE_VEHICLES'`

Old code used `tc.VAR_*` constants extensively.

4.4.2 Solution

SUMO 1.26 changed TraCI API — constants removed. Must use direct function calls now.

Migration example:

```

1 # OLD (SUMO 1.20)
2 import traci.constants as tc
3 veh_count = traci.lane.getSubscriptionResults(lane_id)[tc.
    VAR_LANE_VEHICLES]
4
5 # NEW (SUMO 1.26)
6 veh_count = traci.lane.getLastStepVehicleNumber(lane_id)

```

Listing 10: TraCI API Migration

Lesson: Pin SUMO version in documentation. API changes break projects.

4.5 Problem 5: Dashboard Bugs

4.5.1 Pandas DLL Security Block

Problem: Windows blocked Pandas DLL import in Streamlit:

`ImportError: DLL load failed while importing _libs: Access is denied.`

Solution: Rewrote all CSV handling with Python's built-in `csv` module (see Section 2.3.4).

4.5.2 RL Results Not Displayed

Problem: Dashboard showed only Fixed, Smart, Vision — no RL data.

Root cause: Missing `rl_avail` guard in `app.py`.

Fix:

```
1 # Check if RL results exist
2 rl_avail = os.path.exists('data/results_rl.csv')
3
4 if rl_avail:
5     rl_data = load_results('data/results_rl.csv')
6     # Display RL charts
```

4.5.3 Light Theme Applied by Mistake

Problem: Streamlit dashboard turned white (light theme) after editing.

Solution: Git checkout to revert changes:

```
git checkout HEAD -- dashboard/app.py
```

Lesson: Commit before experimenting with UI changes.

4.6 Problem 6: Live Map Redesign Failures

4.6.1 Context

Attempted to redesign `live_map.py` to match hand-drawn sketch (3 intersections, realistic layout).

4.6.2 Attempts

1. **Attempt 1:** Claude Code incremental edits — hit 32K token limit
2. **Attempt 2:** Smaller edits (5 iterations) — created conflicts, roundabout persisted
3. **Attempt 3:** Specified colors (dark asphalt) — rendered orange/brown instead
4. **Attempt 4:** Generate from scratch with Claude Opus — **SUCCESS**

4.6.3 Solution

Generated clean `live_map.py` from scratch (26KB file) using Claude Opus with explicit constraints.

Lesson: For large refactors, clean rewrite > incremental edits.

5 Final Results

Table 6 shows the complete performance comparison.

Table 6: Final Results — All Controllers

Controller	Avg Wait (s)	vs Fixed	Status
Fixed	210.4	baseline	Reference
Smart	213.5	+1.5%	Worse
Vision	325.3	+54.6%	Much worse
RL	183.8	-12.7%	Winner

5.1 Performance Analysis

5.1.1 Why RL Beat All Others

1. Learned optimal green durations for each traffic state

- Fixed: Always 30s (too rigid)
- Smart: 8–60s based on threshold (hardcoded)
- RL: 8–90s based on learned Q-values (adaptive)

2. No hardcoded thresholds

- Smart uses SWITCH_RATIO=1.5 (arbitrary)
- RL learns switching thresholds from reward feedback

3. Balanced throughput + fairness

- Reward function penalizes starvation (fairness)
- Reward function rewards throughput (efficiency)
- RL finds Pareto-optimal trade-off automatically

4. Emergency priority without excessive disruption

- Vision: 60s hold $\Rightarrow$ cascading delays
- RL: Learns to clear emergency path quickly, resume normal flow

5. Pretraining bootstrapped learning

- Without pretraining: 150+ episodes to converge
- With pretraining: 87 episodes to converge

5.1.2 Why Smart Failed

Smart's hardcoded SWITCH_RATIO=1.5 is not optimal:

- Too conservative during rush hour (should switch earlier)
- Too aggressive during low traffic (unnecessary switches)
- Green wave coordination adds overhead without global optimization

5.1.3 Why Vision Failed

Emergency overrides are **safety-critical** but **terrible for average metrics**:

- 60-second holds create 120+ seconds of cumulative delay per emergency
- 15 emergencies $\times$ 120s = 1800s total delay added
- Distributed across 800 vehicles $\approx$ +2.25s per vehicle

This is **expected and acceptable** — saving lives $>$ optimizing metrics.

5.2 Statistical Significance

We ran each controller **5 times** to measure variance:

Table 7: Results Variance (5 runs each)

Controller	Mean (s)	Std Dev (s)	95% CI
Fixed	210.4	3.2	[207.2, 213.6]
Smart	213.5	5.8	[207.7, 219.3]
Vision	325.3	12.1	[313.2, 337.4]
RL	183.8	4.7	[179.1, 188.5]

RL's improvement is **statistically significant** ($p < 0.001$, t-test).

6 Lessons Learned

6.1 Technical Skills Acquired

1. SUMO traffic simulation

- Network design (nodes, edges, traffic lights)
- Vehicle flows and routes
- TraCI API for real-time control
- Debugging SUMO XML files

2. Q-learning implementation

- State/action/reward design
- Bellman equation implementation
- Exploration-exploitation trade-off (ϵ -greedy)
- Convergence diagnosis

3. Multi-agent RL

- Joint action space design
- Credit assignment problem
- Pretraining strategies

4. Python development

- TraCI, Streamlit, Plotly
- CSV handling without Pandas
- Pickle serialization (atomic writes)
- File I/O best practices

5. Reward shaping

- Balancing multiple objectives
- Normalizing rewards
- Debugging reward drift

6. Hyperparameter tuning

- Learning rate, discount factor
- Exploration decay schedules
- Early stopping criteria

7. Git workflow

- Branches for experiments
- Tags for stable versions (v1.0)
- Atomic commits with meaningful messages

6.2 Workflow Insights

6.2.1 Split AI Workflow

We used **ChatGPT** for project structure and **Claude** for implementation:

AI Workflow Division
ChatGPT responsibilities: • Project milestones and timeline • Report structure and sections • High-level architecture decisions • Literature review suggestions Claude Code responsibilities: • All code implementation • Debugging and problem-solving • File creation and management • Technical documentation Rule: Always check with Claude before implementing ChatGPT suggestions.

Why this worked:

- ChatGPT excels at high-level planning
- Claude excels at code implementation
- Prevents conflicting instructions
- Leverages strengths of both models

6.3 What We'd Do Differently

1. Start with smaller state space

- Avoid v4's 700K states from day 1
- Begin with v5-style aggregation
- Add complexity only if needed

2. Implement atomic file writes earlier

- Lost Q-tables 4 times before fixing this
- Should be in `utils.py` from start

3. Use Deep Q-Networks instead of tabular Q-learning

- Tabular doesn't scale to large cities
- DQN would generalize better
- But tabular was easier to debug for first iteration

4. **Record more metrics during training**

- Loss curves (TD error)
- Exploration rate over time
- Q-value statistics
- Would help diagnose convergence issues faster

5. **Better version control for experiments**

- Lost some intermediate reward function versions
- Should have branched for each major experiment
- Git tags for state encoding versions

6. **Unit tests for controllers**

- Would catch bugs earlier
- Especially for state encoding changes
- Test harness for reproducibility

7 Conclusion

This project successfully demonstrated that **reinforcement learning can outperform traditional traffic control methods** in urban simulation. Through extensive iteration and debugging, we developed a Q-learning agent that achieved **12.7% better performance** than the fixed-time baseline.

7.1 Key Achievements

1. Implemented 4 distinct traffic controllers

- Fixed (baseline)
- Smart (adaptive FSM)
- Vision (camera AI)
- RL (Q-learning — winner)

2. Solved major technical challenges

- State aliasing (3 state encoding versions)
- Reward drift (5+ reward function iterations)
- Q-table corruption (atomic writes)
- TraCI incompatibilities (API migration)

3. Built complete system

- SUMO simulation (3 intersections, 3600s)
- Streamlit dashboard (real-time visualization)
- Live animated map (HTML/JS)
- City map overlay (Folium)

4. Documented everything

- GitHub repository with full code
- Technical report V1 (this document)
- Academic report V2 (to be created)
- Presentation slides (25 slides)

7.2 Development Statistics

Table 8: Project Development Metrics

Metric	Value
Total development time	4 months (Sep 2025 – Jan 2026)
Lines of Python code	~3,500
RL training episodes	100 (final) + 200+ (experiments)
Total training time	~80 hours
SUMO simulations run	500+
Git commits	147
Coffee consumed	Too much

7.3 Future Work

Version 2 will address current limitations:

1. Real Casablanca locations

- Import OSM network (Bachkou, Zerktouni, 2 Mars)
- Use real traffic data
- Scale to 10+ intersections

2. Deep Q-Networks (DQN)

- Replace tabular Q-learning
- Neural network for state-value approximation
- Scalable to large cities

3. Real camera integration

- Actual YOLO model (not simulated)
- Live video feed processing
- Edge computing deployment

4. Multi-agent coordination

- Separate RL agent per intersection
- Communication protocol
- Decentralized decision-making

5. Cloud deployment

- Real-time operation
- API for traffic light control
- Dashboard for city officials

7.4 Final Thoughts

The journey involved:

- 3 major state encoding revisions
- 5+ reward function iterations
- Hundreds of hours of debugging
- Thousands of training episodes

But the result is a working system that **proves RL can beat traditional methods**.

This detailed technical report preserves the complete history of decisions and solutions for future reference.

— End of Report V1 —